

Revisiting randomized choices in isolation forests

Mocanu Ștefan
Voinescu David-Ioan
Chiruș Mina-Sebastian

Final Project of "Anomaly Detection"
course

Facultatea de Matematică și Informatică
Universitatea din București
January 31, 2026

Contents

1	State of the Art	2
2	Algorithm	3
2.1	Node Splitting Mechanisms	3
2.2	Forest Construction Algorithm	4
2.3	Scoring and Anomaly Detection	5
2.4	Complexity Analysis	6
2.4.1	Training Time Complexity	6
2.4.2	Scoring Time Complexity	7
2.4.3	Space Complexity	7
3	Implementation	8
3.1	Models defined	8
3.2	Multi-threading	8
3.2.1	Tree-level Parallelism	8
3.2.2	Prediction Parallelism	8
3.3	Optimizations	8
3.4	Data Structures	9
3.4.1	IsoTree: Standard Node Structure	9
3.4.2	IsoHPlane: Extended/Hyperplane Node	9
3.4.3	IsoForest: Complete Model	9
3.4.4	ExtIsoForest: Extended Model	10
3.5	Design Rationale	10
3.5.1	Flat Vector Storage	10
3.6	Benchmarks	10
3.7	Model Quality (AUROC)	11
4	Experiments	11
4.1	Replicated Results	11
4.1.1	Experimental Protocol	11
4.1.2	Datasets	12
4.1.3	Models	12
4.1.4	Main Replication Results	12
4.1.5	Discussion	13
4.2	Sensitivity Analysis	14
4.2.1	Findings	15
4.3	Reproducibility	16
4.3.1	Environment	16
4.3.2	Running the experiments	16
4.3.3	Outputs	17
4.3.4	Determinism	17

1 State of the Art

This section examines the internal decision logic of prominent isolation-based algorithms. While these methods share the goal of isolating anomalies through recursive partitioning, they differ fundamentally in their **splitting strategies**—specifically in how they select the splitting dimensions, normal vectors, and cut points. The integration of this part of the algorithm is shared with the proposed model from the paper, FCF, and is described in Section 2.2 and Section 2.3.

Isolation Forest(IF)

It uses axis-parallel splits with fully random selection.

Node steps:

- Randomly select a feature j from the set of available dimensions d .
- Randomly select a split value v from a uniform distribution between the minimum and the maximum values of feature j in the current node (S):

$$v \sim U(\min(S_j), \max(S_j))$$

- **Branching Condition:** A data point x is assigned to the left child node if $x_j < v$. Otherwise, it is assigned to the right child.

Extended Isolation Forest(EIF)

It uses random hyperplanes to handle anomalies that are not axis-aligned (oblique splits).

Node steps:

- Generate a random normal vector n from a standard normal distribution $\mathcal{N}(0, 1)$ for each dimension.
- Project the data points onto this normal vector.
- Select a random split point p uniformly within the range of the projected values.
- **Branching Condition:** A point x goes to the left child if:

$$(x - p) \cdot n \leq 0$$

SCiForest(SCiF)

It uses guided hyperplanes. It aims to address the issue where random hyperplanes might pass through sparse regions without actually separating clusters. Unlike EIF, the choice of the normal vector is optimized rather than purely random.

Node steps:

- Generate a set of candidate hyperplanes $\{(n_1, p_1), \dots, (n_k, p_k)\}$ using the EIF approach.
- For each candidate, calculate the **Standard Deviation Gain** ($\Delta\sigma$), which measures the reduction in dispersion:

$$\Delta\sigma = \sigma(S) - \frac{|S_L|}{|S|}\sigma(S_L) - \frac{|S_R|}{|S|}\sigma(S_R)$$

Where $\sigma(S)$ is the standard deviation of the projected points in the current node, and S_L, S_R are the resulting subsets.

- **Selection:** Alternatively, some implementations use a **Density-based** criterion to find sparse regions:

$$Gain = \frac{1}{\text{Density}(S_L) + \text{Density}(S_R)}$$

- Select the specific hyperplane (n_i, p_i) that maximizes this criterion.

2 Algorithm

2.1 Node Splitting Mechanisms

This subsection details the specific decision process occurring inside a single node for FCF.

Uses Constructed Projections and Optimal Splits. Unlike standard Isolation Forest which picks split points randomly, FCF constructs a projection vector z by combining multiple features and then searches for the specific threshold s that best separates the data (minimizing impurity). Node steps:

- Create a projection vector z of size m (number of data points) initialized to zeros.
- Repeat p times (where p is a hyperparameter for the number of splitting variables):
 - Choose a variable index v uniformly at random from the available columns $[1, n]$.
 - Extract the column vector $y = X[:, v]$.
 - Draw a random coefficient c from a Normal distribution $\mathcal{N}(0, 1)$.
 - Update z by adding the standardized version of feature y : $z := z + c \frac{y - \bar{y}}{\sigma_y}$. This step build a random linear combination of vectors.
- Instead of picking a random cut, iterate through possible values of s in z to find the one that minimizes the cluster impurity. i.e. choose s from z that minimizes:

$$\frac{m_{\text{left}}\sigma_{z_l} + m_{\text{right}}\sigma_{z_r}}{m_{\text{left}} + m_{\text{right}}}$$

- Divide the data X into left set X_l (where $z_i \leq s$) and right set X_r (where $z_i > s$).

The Gain criterion is:

$$G = \frac{n_L}{n} \sigma(v_{z_L}) + \frac{n_R}{n} \sigma(v_{z_R})$$

Where:

1. m : Number of data points (rows) in the current node.
2. n : Number of variables (columns) in the dataset.
3. z : The constructed projection vector (size m).
4. z_l, z_r : The subsets of z values falling into the left and right children.
5. $m_{\text{left}}, m_{\text{right}}$: The number of points in the left and right children.
6. σ_z : The standard deviation of the values in vector z .

2.2 Forest Construction Algorithm

The construction of the Fair-Cut Forest (FCF) follows an ensemble approach similar to the standard Isolation Forest but incorporates the specific optimized splitting mechanism defined in the node training phase.

The algorithm takes as input the dataset $\mathbf{X}$ of size $m \times n$, the number of trees t , the sub-sampling size s , and the number of splitting variables p .

1. **Sub-sampling**: The algorithm iterates t times to build the forest. In each iteration, a subset $\mathbf{X}_s$ is created by selecting s data points uniformly at random without replacement from the original dataset $\mathbf{X}$. This sub-sampling (typically $s = 256$) is critical for maintaining diversity among the trees and preventing overfitting.
2. **Tree Growth**: For each subset $\mathbf{X}_s$, a tree is grown using the `FairCutTree` process (described in Section A).
 - The root node is initialized with the subset $\mathbf{X}_s$ and current depth $d = 0$.
 - The tree recursively partitions the data using the optimized gain criterion until the constraints (max depth or single data point) are met.
3. **Normalization Factor ($c(s)$)**: Once the forest is built, the algorithm calculates the expected path length of an unsuccessful search in a Binary Search Tree (BST) for a dataset of size s . This value, denoted as $c(s)$, serves as the baseline for normalization:

$$c(s) = 2H(s-1) - \frac{2(s-1)}{s}$$

where $H(i)$ is the harmonic number, estimated as $\ln(i) + 0.5772156649$ (Euler's constant).

4. **Output**: The result is a set $\mathbb{T}$ containing t trained Fair-Cut Trees and the normalization factor $c(s)$.

2.3 Scoring and Anomaly Detection

The scoring phase determines the degree of anomaly for a new query point $\mathbf{x}$. This process involves traversing the trees using the parameters learned during training and aggregating the path lengths.

1. Tree Traversal and Projection Reconstruction

To calculate the path length $h(\mathbf{x})$ in a single tree, the point $\mathbf{x}$ must traverse from the root to a leaf. Because FCF uses composite projections, the splitting value z must be mathematically reconstructed at each node using the stored training parameters.

- **Initialization:** Upon entering a node, the projection value z is initialized to 0.
- **Reconstruction Loop:** The algorithm iterates through the specific set of variables $v \in \{1, \dots, p\}$ that were selected during the training of that node. For each variable, the projection is updated:

$$z := z + c_v \frac{x_v - \bar{y}_v}{\sigma_{y_v}}$$

Here, x_v is the value of the feature in the query point, while c_v (random coefficient), $\bar{y}_v$ (mean), and σ_{y_v} (standard deviation) are the specific values stored in the node during training. This ensures the point is projected onto the exact same hyperplane used to define the split.

- **Branching Decision:** The reconstructed z is compared to the optimal split threshold s stored in the node:
 - If $z \leq s$, the traversal continues to the Left child.
 - If $z > s$, the traversal continues to the Right child.
- **Termination:** This recursive process continues until a leaf node is reached. The returned value is the depth of the leaf (the number of edges from the root).

The final anomaly score $S(\mathbf{x}, s)$ is derived by averaging the path lengths across the entire forest $\mathbb{T}$.

1. **Average Path Length:** The query point $\mathbf{x}$ is passed through all t trees. The average path length $E(h(\mathbf{x}))$ is calculated as:

$$E(h(\mathbf{x})) = \frac{1}{t} \sum_{i=1}^t h_i(\mathbf{x})$$

2. **Score Normalization:** The average path length is normalized by the expected path length $c(s)$ to produce a score between 0 and 1:

$$S(\mathbf{x}, s) = 2^{-\frac{E(h(\mathbf{x}))}{c(s)}}$$

3. Interpretation:

- $S \rightarrow 1$: The point has a short average path length, indicating it was easily isolated. It is likely an **anomaly**.
- $S \rightarrow 0.5$: The point has an average path length, indicating it is likely a normal observation.
- $S \rightarrow 0$: The point is deep inside the cluster, indicating it is very normal.

2.4 Complexity Analysis

This section analyzes the computational complexity of the Fair-Cut Forest (FCF) and contrasts it with the standard Isolation Forest (IF) and Extended Isolation Forest (EIF). The primary trade-off proposed by the FCF model is accepting higher computational costs per node in exchange for optimized splits that better isolate clustered anomalies.

2.4.1 Training Time Complexity

The training complexity is determined by the cost of constructing the trees. For a forest of t trees and a sub-sample size of s :

- **Isolation Forest (IF)**: The standard IF selects a feature and a split value uniformly at random. This requires $\mathcal{O}(1)$ operations per node (constant time). The total complexity for building the forest is:

$$\mathcal{O}(t \cdot s \log s)$$

- **Fair-Cut Forest (FCF)**: In FCF, the splitting process is significantly more involved. At each node containing m data points:
 1. **Projection Construction**: The algorithm iterates p times (the number of splitting variables). In each iteration, it performs vector operations on m points to update $\mathbf{z}$. This costs $\mathcal{O}(m \cdot p)$.
 2. **Split Optimization**: To find the optimal split s , the projection vector $\mathbf{z}$ is typically sorted, costing $\mathcal{O}(m \log m)$.

Consequently, the total training complexity for FCF is:

$$\mathcal{O}(t \cdot s \cdot (p + \log s) \cdot \log s)$$

Since p is a hyperparameter often set > 1 , FCF is computationally more expensive than IF. However, the paper argues that this cost is justified as it leads to more efficient isolation (shallower trees for anomalies).

2.4.2 Scoring Time Complexity

The scoring complexity refers to the time required to evaluate a new query point $\mathbf{x}$.

- **Isolation Forest (IF):** A point simply traverses the tree by making single-feature comparisons. The cost is proportional to the tree height ($\log s$).

$$\mathcal{O}(t \cdot \log s)$$

- **Fair-Cut Forest (FCF):** As described in Section 2.3, traversing a node requires reconstructing the projection z .
 - At each node in the path, the algorithm must perform p updates (one for each splitting variable stored during training).
 - Therefore, the cost per node is $\mathcal{O}(p)$.

The total scoring complexity is:

$$\mathcal{O}(t \cdot p \cdot \log s)$$

This makes prediction slower than IF by a factor of p , but comparable to Extended Isolation Forest (EIF) if p is close to the data dimensionality n .

2.4.3 Space Complexity

The memory requirement is defined by the parameters stored in the trained model.

- **Isolation Forest (IF):** Stores only the split dimension index and split value per node. Space is minimal: $\mathcal{O}(t \cdot s)$.
- **Fair-Cut Forest (FCF):** For each internal node, FCF must store the parameters required to reconstruct the projection. This includes:
 - The indices of the p variables used.
 - The p random coefficients (c_v).
 - The p means ($\bar{y}_v$) and standard deviations (σ_{y_v}).

Thus, the space complexity scales linearly with p :

$$\mathcal{O}(t \cdot s \cdot p)$$

3 Implementation

3.1 Models defined

- **iForest**: Original Isolation Forest (Liu et al., 2008)
- **EIF**: Extended Isolation Forest (Hariri et al., 2019)
- **SCiForest**: Split-Criterion Isolation Forest (Liu et al., 2010)
- **FCF**: Fair-Cut Forest (Cortes, 2021)
- **RRCF**: Robust Random Cut Forest (Guha et al., 2016)

3.2 Multi-threading

3.2.1 Tree-level Parallelism

```
for (size_t tree = 0; tree < ntrees; tree++) {  
    // Build tree independently  
    build_tree(tree, sample, model);  
}
```

Each tree is built independently, allowing embarrassingly parallel construction.

3.2.2 Prediction Parallelism

```
for (size_t i = 0; i < n_observations; i++) {  
    scores[i] = score_observation(X[i], forest);  
}
```

Predictions for different observations are computed in parallel.

3.3 Optimizations

The library supports:

- **-march=native**: Compile-time CPU optimization (AVX, AVX2, etc.)
- **Cache-friendly data layouts**: Flat vector storage instead of pointer-based trees
- **Contiguous memory access**: Improves cache locality during tree traversal

3.4 Data Structures

3.4.1 IsoTree: Standard Node Structure

```
typedef struct IsoTree {
    ColType    col_type;           // Numeric, Categorical, or NotUsed
    size_t     col_num;           // Column index for split
    double     num_split;         // Split threshold (numeric)
    std::vector<char> cat_split;   // Bitmap for categorical splits
    int        chosen_cat;        // Selected category
    size_t     tree_left;         // Index to left child
    size_t     tree_right;        // Index to right child
    double     pct_tree_left;     // Fraction going left
    double     score;             // Depth/score at node
    double     range_low;         // Range bounds for penalization
    double     range_high;
    double     remainder;         // For distance calculations
} IsoTree;
```

3.4.2 IsoHPlane: Extended/Hyperplane Node

For extended isolation forest with hyperplane splits:

```
typedef struct IsoHPlane {
    std::vector<size_t> col_num;   // Multiple columns
    std::vector<ColType> col_type; // Type per column
    std::vector<double> coef;      // Linear combination coeffs
    std::vector<double> mean;      // Standardization means
    std::vector<double> fill_val;  // Missing value handling
    std::vector<std::vector<double>> cat_coef; // Categorical coeffs

    double     split_point;        // Threshold on combination
    size_t     hplane_left;        // Left child index
    size_t     hplane_right;       // Right child index
    double     score;
    double     range_low, range_high;
    double     remainder;
} IsoHPlane;
```

3.4.3 IsoForest: Complete Model

```
typedef struct IsoForest {
    std::vector<std::vector<IsoTree>> trees; // Forest storage
    NewCategAction new_cat_action;           // Handle new categories
}
```

```

    CategSplit      cat_split_type;    // Categorical strategy
    MissingAction    missing_action;    // Missing value handling
    ScoringMetric    scoring_metric;    // depth, density, etc.
    double           exp_avg_depth;     // Expected isolation depth
    double           exp_avg_sep;       // Expected separation
    size_t           orig_sample_size;  // Training sample size
    bool             has_range_penalty;
} IsoForest;

```

3.4.4 ExtIsoForest: Extended Model

```

typedef struct ExtIsoForest {
    std::vector<std::vector<IsoHPlane>> hplanes; // Hyperplane trees
    // Same metadata as IsoForest...
} ExtIsoForest;

```

3.5 Design Rationale

3.5.1 Flat Vector Storage

Trees are stored as **flat vectors** rather than traditional pointer-based nodes:

- **Cache efficiency:** Contiguous memory layout
- **Serialization:** Easy to save/load models
- **Thread safety:** No pointer aliasing issues
- **Memory locality:** Children accessed by index, not pointer

```

// Tree traversal by index
size_t current = 0; // Root
while (!is_terminal(tree[current])) {
    if (x[tree[current].col_num] <= tree[current].num_split)
        current = tree[current].tree_left;
    else
        current = tree[current].tree_right;
}

```

3.6 Benchmarks

From the isotree repository benchmarks (100 trees, CovType dataset):

Library	Model	256 samples	1024 samples	10k samples
isotree	orig	0.00161s	0.00631s	0.0848s
isotree	ext	0.00326s	0.0123s	0.168s
eif	orig	0.149s	0.398s	4.99s
eif	ext	0.16s	0.428s	5.06s
H2O	orig	9.33s	11.21s	14.23s
H2O	ext	1.06s	2.07s	17.31s
scikit-learn	orig	8.30s	8.01s	6.89s
solitude	orig	32.61s	34.01s	41.01s

Table 1: Training time comparison on CovType dataset, 100 trees (lower is better)

Key observation: isotree is typically 1–2 orders of magnitude faster than other implementations.

3.7 Model Quality (AUROC)

Library	AUROC (defaults)	AUROC (grid search)
isotree	0.70	0.84
eif	–	0.714
scikit-learn	0.687	0.74
H2O	0.662	0.748

Table 2: Outlier detection quality on Satellite dataset (higher is better)

Library	AUROC (defaults)	AUROC (grid search)
isotree	0.80	0.982
eif	–	0.808
scikit-learn	0.836	0.836
H2O	0.80	0.80

Table 3: Outlier detection quality on Annthyroid dataset (higher is better)

4 Experiments

4.1 Replicated Results

4.1.1 Experimental Protocol

We follow the paper’s evaluation protocol for isolation-forest-style methods: for each dataset, we (i) fit each model in an unsupervised manner on the full dataset X , (ii)

Table 4: Datasets used in the partial replication (subset of the paper’s Table 5).

Dataset	n	d	Outliers (%)
Arrhythmia	452	274	14.60
Pima	768	8	34.90
SpamBase	4601	57	39.40
Satellite	6435	36	31.64
Pendigits	6870	16	2.27
Anthyroid	7200	6	7.42
MNIST	7603	100	9.21

compute anomaly scores for the same full dataset, and (iii) evaluate against the provided binary labels using ROC-AUC and PR-AUC. All results are averaged over 10 random seeds (seeds 0–9), reporting mean $\pm$ standard deviation.

4.1.2 Datasets

The original paper reports results on nine benchmark datasets (Table 5 in the original paper). In this project we perform a *partial replication* on a 7/9 subset due to data/time constraints. Table 4 summarizes the datasets used and the fraction of outliers.

4.1.3 Models

We evaluate the isolation-forest family variants emphasized in the paper: **IF** (standard isolation forest), **IF-u** (higher-capacity variant with more trees and unlimited depth), **EIF** (extended splits with $ndim = 2$ under random splitting), **SCiF** and **SCiF-u** (guided splits using average gain with range penalty enabled), and **FCF** (pooled-gain guided splitting; Fair-Cut-style). All methods are implemented using the authors’ **isotree** library with parameters selected to match the intended definitions of these model families.

The paper reports two EIF implementations (EIF-o and EIF-t); in this project we use a single EIF-style configuration ($ndim=2$ random hyperplane splits) from the authors’ isotree library.

4.1.4 Main Replication Results

Tables 5 and 6 report ROC-AUC and PR-AUC, respectively, as mean $\pm$ std over 10 seeds. Within each dataset row we highlight the best performing method in bold. These tables constitute the core replicated results under the paper-style protocol.

For the overlapping datasets/models, our ROC/PR are in the same range as the paper and preserve the qualitative rankings (e.g., FCF strong on Satellite; SCiF-u strong on Anthyroid). Differences are expected due to implementation differences (single EIF config vs EIF-o/EIF-t) and dataset preprocessing.

Table 5: ROC-AUC (mean $\pm$ std over 10 seeds). Best per row in bold.

Dataset	IF	IF-u	EIF	SCiF	SCiF-u	FCF
Arrhythmia	0.8054 $\pm$ 0.0016	0.8081$\pm$0.0005	0.7817 $\pm$ 0.0008	0.7004 $\pm$ 0.0007	0.7388 $\pm$ 0.0004	0.7893 $\pm$ 0.0004
Pima	0.6950 $\pm$ 0.0021	0.6734 $\pm$ 0.0008	0.6852 $\pm$ 0.0011	0.6150 $\pm$ 0.0005	0.6580 $\pm$ 0.0002	0.7356$\pm$0.0005
SpamBase	0.6403 $\pm$ 0.0043	0.7134 $\pm$ 0.0008	0.5858 $\pm$ 0.0046	0.4696 $\pm$ 0.0011	0.4268 $\pm$ 0.0002	0.7163$\pm$0.0011
Satellite	0.6929 $\pm$ 0.0060	0.7053 $\pm$ 0.0008	0.6802 $\pm$ 0.0021	0.6041 $\pm$ 0.0021	0.7450 $\pm$ 0.0003	0.8151$\pm$0.0007
Pendigits	0.9587 $\pm$ 0.0025	0.9500 $\pm$ 0.0010	0.9621 $\pm$ 0.0011	0.9819 $\pm$ 0.0006	0.9844$\pm$0.0002	0.9529 $\pm$ 0.0007
Annnthyroid	0.7963 $\pm$ 0.0037	0.8334 $\pm$ 0.0011	0.8177 $\pm$ 0.0041	0.7928 $\pm$ 0.0002	0.9661$\pm$0.0001	0.8613 $\pm$ 0.0006
MNIST	0.8236 $\pm$ 0.0021	0.8035 $\pm$ 0.0009	0.8385 $\pm$ 0.0049	0.8586$\pm$0.0021	0.8443 $\pm$ 0.0011	0.7934 $\pm$ 0.0005

Table 6: PR-AUC (mean $\pm$ std over 10 seeds). Best per row in bold.

Dataset	IF	IF-u	EIF	SCiF	SCiF-u	FCF
Arrhythmia	0.4409 $\pm$ 0.0061	0.4755$\pm$0.0031	0.4570 $\pm$ 0.0032	0.3276 $\pm$ 0.0015	0.3540 $\pm$ 0.0020	0.4482 $\pm$ 0.0051
Pima	0.5127 $\pm$ 0.0018	0.5000 $\pm$ 0.0009	0.5050 $\pm$ 0.0014	0.4250 $\pm$ 0.0004	0.4578 $\pm$ 0.0003	0.5596$\pm$0.0007
SpamBase	0.4884 $\pm$ 0.0049	0.5544$\pm$0.0014	0.4429 $\pm$ 0.0042	0.3778 $\pm$ 0.0007	0.3478 $\pm$ 0.0002	0.5466 $\pm$ 0.0015
Satellite	0.6410 $\pm$ 0.0024	0.6603 $\pm$ 0.0008	0.6144 $\pm$ 0.0022	0.5871 $\pm$ 0.0011	0.6822 $\pm$ 0.0006	0.7236$\pm$0.0015
Pendigits	0.2958 $\pm$ 0.0106	0.2765 $\pm$ 0.0059	0.2616 $\pm$ 0.0064	0.4197 $\pm$ 0.0095	0.4297$\pm$0.0029	0.2493 $\pm$ 0.0042
Annnthyroid	0.2755 $\pm$ 0.0034	0.3075 $\pm$ 0.0021	0.2985 $\pm$ 0.0048	0.5057 $\pm$ 0.0008	0.6558$\pm$0.0003	0.3737 $\pm$ 0.0009
MNIST	0.2944 $\pm$ 0.0028	0.2774 $\pm$ 0.0024	0.3349 $\pm$ 0.0097	0.4000$\pm$0.0034	0.3732 $\pm$ 0.0025	0.2751 $\pm$ 0.0013

4.1.5 Discussion

Several trends match the qualitative conclusions of the paper.

No single method dominates across all datasets We observe dataset-dependent behavior: FCF performs best on Pima and Satellite (both ROC-AUC and PR-AUC), while SCiF-u is strongest on Pendigits and Annnthyroid, and SCiF is best on MNIST. This supports the paper’s central message that different randomized/guided splitting choices can be beneficial depending on the data distribution.

Higher-capacity variants help selectively Increasing capacity (more trees and unlimited depth, the “-u” variants) improves Arrhythmia for IF and strongly boosts SCiF on Satellite and Annnthyroid. However, IF-u slightly underperforms IF on MNIST, indicating that additional capacity is not universally beneficial under the full-data evaluation protocol.

Guided splitting can help or hurt depending on the dataset SCiF/SCiF-u excel on Pendigits, Annnthyroid, and MNIST, but perform poorly on SpamBase relative to IF-u and FCF. This suggests that gain-guided splitting may amplify dataset-specific artifacts: it can isolate informative structure in some datasets, while leading to suboptimal partitions in others.

Table 7: Sensitivity results on **Satellite** (6435×36 , 31.64% outliers). Mean $\pm$ std over 10 seeds.

Sweep	Configuration	ROC-AUC	PR-AUC	Time (s)
IF depth & trees	IF_t100_d8	0.6929 $\pm$ 0.0060	0.6410 $\pm$ 0.0024	0.028
	IF_t200_d8	0.6872 $\pm$ 0.0030	0.6381 $\pm$ 0.0012	0.062
	IF_t100_dNone	0.6955 $\pm$ 0.0024	0.6410 $\pm$ 0.0020	0.087
	IF_t200_dNone	0.7053 $\pm$ 0.0008	0.6603 $\pm$ 0.0008	0.171
IF sample_size	IF_s64	0.6768 $\pm$ 0.0017	0.6648 $\pm$ 0.0020	0.030
	IF_s128	0.7011 $\pm$ 0.0062	0.6830 $\pm$ 0.0039	0.029
	IF_s256	0.6929 $\pm$ 0.0060	0.6410 $\pm$ 0.0024	0.028
	IF_s512	0.6966 $\pm$ 0.0028	0.6481 $\pm$ 0.0032	0.028
Split geometry	ndim1_random	0.6929 $\pm$ 0.0060	0.6410 $\pm$ 0.0024	0.029
	ndim2_random	0.6949 $\pm$ 0.0028	0.6490 $\pm$ 0.0028	0.084
FCF trees & sample_size	FCF_t100_s256	0.8173 $\pm$ 0.0006	0.7267 $\pm$ 0.0011	0.120
	FCF_t200_s256	0.8246 $\pm$ 0.0005	0.7356 $\pm$ 0.0009	0.264
	FCF_t200_s128	0.8373$\pm$0.0006	0.7660$\pm$0.0013	0.186
	FCF_t200_s512	0.8139 $\pm$ 0.0008	0.7020 $\pm$ 0.0019	0.371
SCiF <i>ntry</i> & range penalty	SCiF_ntry1_pen0	0.6116 $\pm$ 0.0039	0.6163 $\pm$ 0.0030	0.078
	SCiF_ntry10_pen0	0.6082 $\pm$ 0.0051	0.5890 $\pm$ 0.0031	0.208
	SCiF_ntry10_pen1	0.6104 $\pm$ 0.0049	0.5910 $\pm$ 0.0031	0.211
Scoring metric (IF)	IF_score_depth	0.6929 $\pm$ 0.0060	0.6410 $\pm$ 0.0024	0.028
	IF_score_density	0.8324$\pm$0.0020	0.7574$\pm$0.0020	0.029

PR-AUC is especially informative for rare outliers On Pendigits (2.27% outliers), ROC-AUC is high for multiple methods, yet PR-AUC distinguishes the guided methods (SCiF/SCiF-u) more clearly from the random-split baselines. This aligns with the motivation for reporting both ROC-AUC and PR-AUC in anomaly detection.

4.2 Sensitivity Analysis

In addition to replicating the main comparisons, we analyze the sensitivity of the isolation-forest family to key hyperparameters that control (i) model capacity (number of trees and depth limit), (ii) locality and stability (subsample size), (iii) split geometry (axis-aligned vs random hyperplane splits), (iv) the strength of guidance in SCiForest (*ntry* and range penalty), and (v) the scoring rule (depth vs density). All sensitivity results follow the same evaluation protocol as in Section 4.1: unsupervised fit on the full dataset, scoring on the full dataset, and mean $\pm$ std over 10 random seeds (0–9).

To keep the presentation compact, we show full sweep tables for two representative datasets (**Satellite** and **Pendigits**), plus one smaller table highlighting the scoring-metric effect across all datasets. The full CSV outputs for all datasets are included in the repository.

Table 8: Sensitivity results on **Pendigits** (6870×16 , 2.27% outliers). Mean $\pm$ std over 10 seeds.

Sweep	Configuration	ROC-AUC	PR-AUC	Time (s)
IF depth & trees	IF_t100_d8	0.9587 $\pm$ 0.0025	0.2958 $\pm$ 0.0106	0.033
	IF_t200_d8	0.9625$\pm$0.0006	0.3134$\pm$0.0030	0.070
	IF_t100_dNone	0.9465 $\pm$ 0.0033	0.2741 $\pm$ 0.0117	0.080
	IF_t200_dNone	0.9500 $\pm$ 0.0010	0.2765 $\pm$ 0.0059	0.171
IF sample_size	IF_s64	0.9201 $\pm$ 0.0032	0.2157 $\pm$ 0.0063	0.037
	IF_s128	0.9401 $\pm$ 0.0007	0.2722 $\pm$ 0.0120	0.034
	IF_s256	0.9587$\pm$0.0025	0.2958$\pm$0.0106	0.033
	IF_s512	0.9416 $\pm$ 0.0011	0.2675 $\pm$ 0.0058	0.032
Split geometry	ndim1_random	0.9587$\pm$0.0025	0.2958$\pm$0.0106	0.033
	ndim2_random	0.9511 $\pm$ 0.0016	0.2212 $\pm$ 0.0060	0.093
FCF trees & sample_size	FCF_t100_s256	0.9640 $\pm$ 0.0022	0.2444 $\pm$ 0.0105	0.129
	FCF_t200_s256	0.9611 $\pm$ 0.0007	0.2295 $\pm$ 0.0016	0.256
	FCF_t200_s128	0.9667$\pm$0.0009	0.2741$\pm$0.0029	0.197
	FCF_t200_s512	0.9501 $\pm$ 0.0007	0.1938 $\pm$ 0.0021	0.419
SCiF <i>ntry</i> & range penalty	SCiF_ntry1_pen0	0.9588 $\pm$ 0.0010	0.2347 $\pm$ 0.0050	0.092
	SCiF_ntry10_pen0	0.9768$\pm$0.0008	0.4977$\pm$0.0114	0.191
	SCiF_ntry10_pen1	0.9764 $\pm$ 0.0008	0.4382 $\pm$ 0.0082	0.190
Scoring metric (IF)	IF_score_depth	0.9587$\pm$0.0025	0.2958$\pm$0.0106	0.033
	IF_score_density	0.9493 $\pm$ 0.0011	0.2629 $\pm$ 0.0033	0.033

4.2.1 Findings

Capacity is not monotonic Increasing the number of trees and removing the depth limit can yield strong gains (e.g., SpamBase improves from ROC 0.64 to 0.71 and PR 0.49 to 0.55 when moving from 100 trees with depth 8 to 200 trees with unlimited depth), but can also degrade performance (e.g., MNIST decreases from ROC 0.824 to 0.804 under the same change). This suggests that additional capacity amplifies the inductive bias of the splitting strategy: it helps when induced partitions align with the anomaly structure, but can drift otherwise.

Subsample size controls locality The subsample size (`sample_size`) behaves as a locality knob with a dataset-dependent optimum. For instance, on Arrhythmia, smaller subsamples (64–128) increase PR-AUC relative to 256, while on MNIST and Pendigits too-small subsamples (64–128) reduce both ROC-AUC and PR-AUC. Thus, `sample_size` changes the effective detection regime rather than only affecting computational cost.

Split geometry has mixed impact Changing from axis-aligned splits to random hyperplane splits ($ndim = 2$) can yield modest gains on some datasets (e.g., Arrhythmia PR-AUC increases from 0.441 to 0.476) but can degrade others (e.g., Annthyroid ROC-AUC drops from 0.796 to 0.716). This supports the paper’s main message that randomized choices should be treated as modeling decisions rather than fixed defaults.

Table 9: Effect of scoring metric for IF: depth vs density (mean over 10 seeds). Density helps strongly on Satellite/SpamBase/Annthyroid, but is worse on MNIST/Pendigits/Arrhythmia.

Dataset	PR-AUC (depth)	PR-AUC (density)
Arrhythmia	0.4409	0.4148
Pima	0.5127	0.5337
SpamBase	0.4884	0.5470
Satellite	0.6410	0.7574
Pendigits	0.2958	0.2629
Annthyroid	0.2755	0.3321
MNIST	0.2944	0.2587

Guidance strength dominates SCiForest behavior The number of candidate splits (`ntry`) has a large effect in SCiForest: for Pendigits, increasing `ntry` from 1 to 10 yields a large PR-AUC gain ($0.235 \rightarrow 0.498$), while for Arrhythmia it reduces performance substantially (PR-AUC $0.534 \rightarrow 0.422$). The range penalty (`penalize_range`) has a smaller effect on most datasets, although it can reduce PR-AUC in rare-outlier settings.

Scoring metric matters Replacing depth-based scoring with density-based scoring for IF yields large improvements on some datasets (e.g., Satellite PR-AUC $0.641 \rightarrow 0.757$), while reducing performance on others (e.g., MNIST PR-AUC $0.294 \rightarrow 0.259$). This indicates that the scoring rule can change the effective notion of “isolatability” and should be treated as a modeling choice.

4.3 Reproducibility

We provide a fully reproducible implementation of all experiments in a public repository accompanying this report. All numerical results in Sections 4.1 and 4.2 are generated by the code in the repository, and are exported as CSV files that can be used to re-create the tables.

4.3.1 Environment

Experiments were run using Python (3.10+) with the following main dependencies: `numpy`, `pandas`, `scikit-learn`, `scipy`, and the authors’ `isotree` library (Cortes). A `requirements.txt` file is included to recreate the environment.

4.3.2 Running the experiments

All experiments use fixed seeds `0..9`. The main scripts are:

Replication runs (Section 4.1) This script evaluates IF/IF-u/EIF/SCiF/SCiF-u/FCF (plus baselines) on each dataset and writes a CSV file:

```
python replicate_paper.py
# output: anomaly_detection_results.csv
```

Sensitivity runs (Section 4.2) This script performs the parameter sweeps (capacity, sample size, split geometry, guidance, scoring metric) and writes a CSV file:

```
python sensitivity.py
# output: parameter_sensitivity.csv
```

4.3.3 Outputs

The scripts produce two CSV files:

- `anomaly_detection_results.csv` — main replication results.
- `parameter_sensitivity.csv` — sensitivity sweep results.

All tables shown in the report are obtained by using these CSV files (mean and standard deviation across seeds).

4.3.4 Determinism

To reduce variability, we fix the random seed of each run and average over 10 seeds. For LOF/OCSVM baselines, results are deterministic under a fixed preprocessing pipeline and therefore have near-zero variation across repeated runs. Runtime measurements report wall-clock time for model fitting and scoring (excluding dataset loading).